



Security Assessment

Altitude

Verified on 08/12/2026

SUMMARY

Project

Altitude

CHAIN

Base

METHODOLOGY

Manual & Automatic Analysis

FILES

Repository

DELIVERY

08/12/2026

TYPE

Standard Audit



Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

0 Critical

An exposure that can affect the contract functions in several events that can risk and disrupt the contract

0 Major

An opening & exposure to manipulate the contract in an unwanted manner

3 Medium

An opening that could affect the outcome in executing the contract in a specific situation

0 Minor

An opening but doesn't have an impact on the functionality of the contract

1 Informational

An opening that consists information but will not risk or affect the contract

0 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS
✓ AUDIT PASSED

TABLE OF CONTENTS | Altitude

| Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

| Project Information

Token/Project Details
Inheritance Graph
Call Graph

| Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

DISCLAIMER | Altitude

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

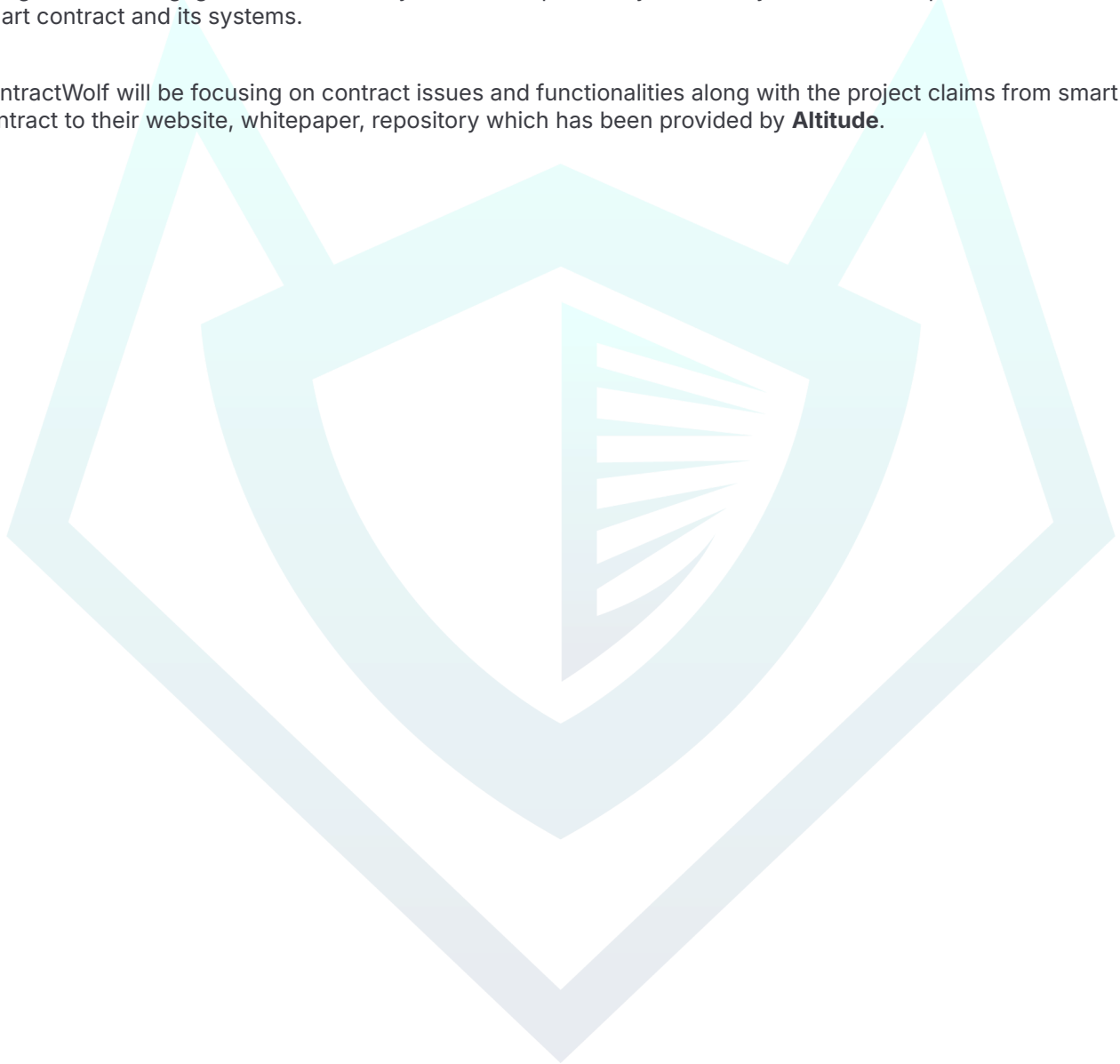
Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | Altitude

Altitude team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Altitude**.



AUDITING APPROACH | Altitude

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | Altitude



Altitude is a project built on Base. The Altitude token is the core of The Altitude Project.

Token Name

-

Symbol

-

Decimal

-

Total Supply

-

Chain

Base

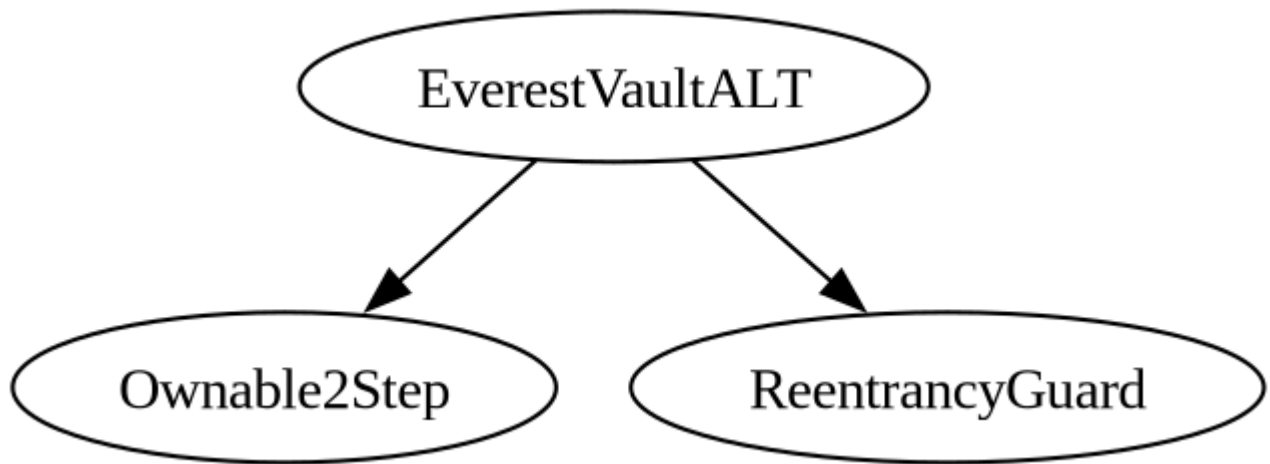
SOURCE

Source

Sent Via local-files

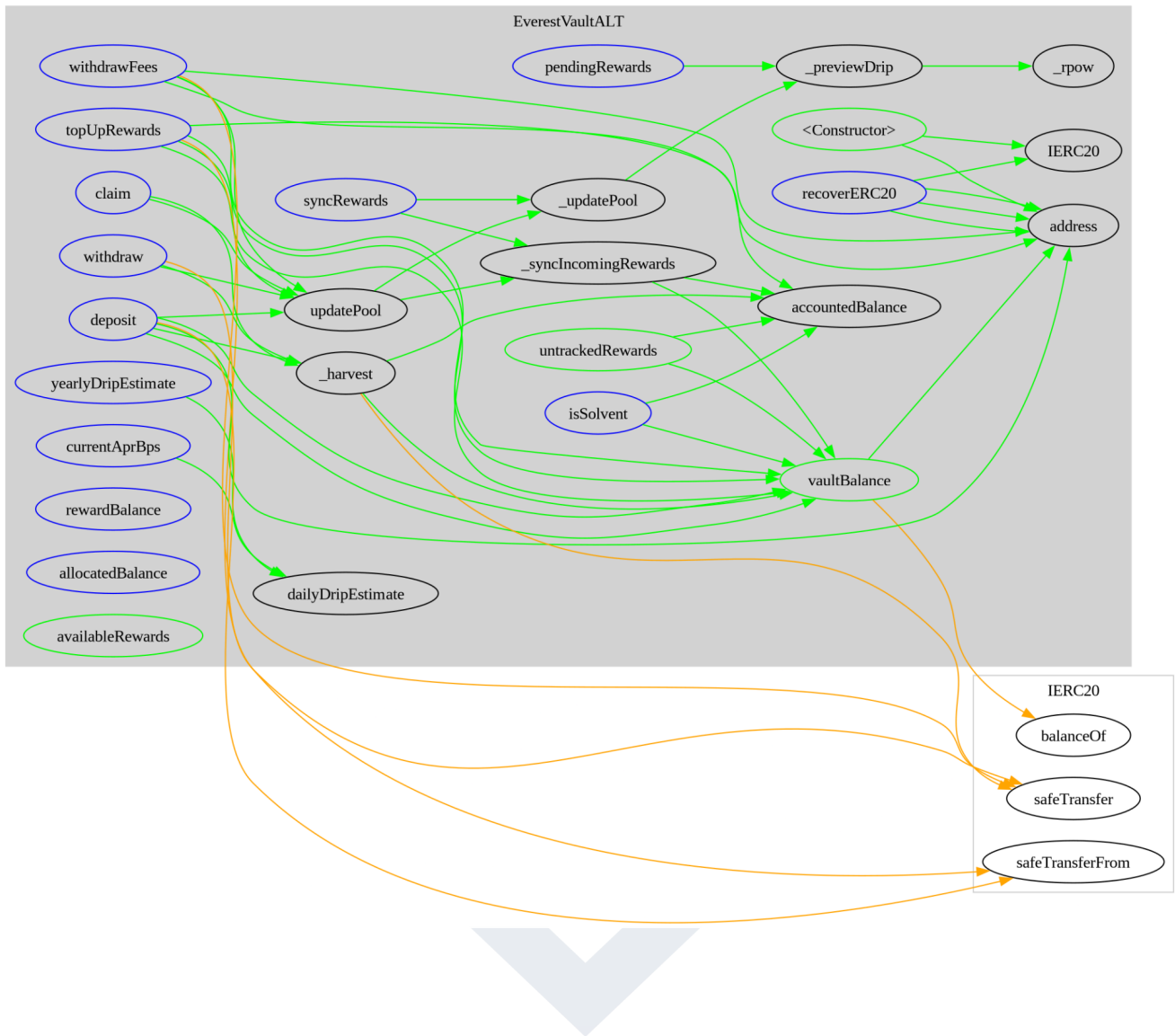
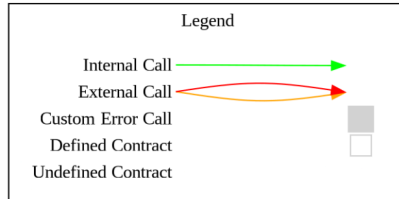
INHERITANCE GRAPH | Altitude

Inheritance Graph of Contract Functions



CALL GRAPH | Altitude

Call Graph of Contract Functions



FINDINGS | Altitude



Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

This report has been prepared to state the issues and vulnerabilities for Altitude through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
N/A	Rounding drift locks principal	EverestVaultAlt.sol, L: 238, 288, 339	Medium	• Pending
N/A	A one-wei staker permanently caps the vault	EverestVaultAlt.sol, L: 238, 288	Medium	• Pending
N/A	A dust staker collects all the rewards	EverestVaultAlt.sol, L: 231, 238	Medium	• Pending
N/A	Renouncing ownership strands the fees	EverestVaultAlt.sol, L: 356	Informational	• Pending

SWC ATTACKS | Altitude

Smart Contract Weakness Classification and Test Cases

ID	Description	Status
SWC-100	Function Default Visibility	● Passed
SWC-101	Integer Overflow and Underflow	● Passed
SWC-102	Outdated Compiler Version	● Passed
SWC-103	Floating Pragma	● Passed
SWC-104	Unchecked Call Return Value	● Passed
SWC-105	Unprotected Ether Withdrawal	● Passed
SWC-106	Unprotected SELF DESTRUCT Instruction	● Passed
SWC-107	Reentrancy	● Passed
SWC-108	State Variable Default Visibility	● Passed
SWC-109	Uninitialized Storage Pointer	● Passed
SWC-110	Assert Violation	● Passed
SWC-111	Use of Deprecated Solidity Functions	● Passed
SWC-112	Delegate call to Untrusted Callee	● Passed
SWC-113	DoS with Failed Call	● Passed
SWC-114	Transaction Order Dependence	● Passed
SWC-115	Authorization through tx.origin	● Passed
SWC-116	Block values as a proxy for time	● Passed
SWC-117	Signature Malleability	● Passed
SWC-118	Incorrect Constructor Name	● Passed
SWC-119	Shadowing State Variables	● Passed
SWC-120	Weak Sources of Randomness from Chain Attributes	● Passed
SWC-121	Missing Protection against Signature Replay Attacks	● Passed
SWC-122	Lack of Proper Signature Verification	● Passed

ID	Description	Status
SWC-123	Requirement Violation	● Passed
SWC-124	Write to Arbitrary Storage Location	● Passed
SWC-125	Incorrect Inheritance Order	● Passed
SWC-126	Insufficient Gas Griefing	● Passed
SWC-127	Arbitrary Jump with Function Type Variable	● Passed
SWC-128	DoS With Block Gas Limit	● Passed
SWC-129	Typographical Error	● Passed
SWC-130	Right-To-Left-Override control character(U+202E)	● Passed
SWC-131	Presence of unused variables	● Passed
SWC-132	Unexpected Ether balance	● Passed
SWC-133	Hash Collisions With Multiple Variable Arguments	● Passed
SWC-134	Message call with hardcoded gas amount	● Passed
SWC-135	Code With No Effects	● Passed
SWC-136	Unencrypted Private Data On-Chain	● Passed

CW ASSESSMENT | Altitude

ContractWolf Vulnerability and Security Tests

ID	Name	Description	Status
CW-001	Multiple Version	Presence of multiple compiler version across all contracts	✓
CW-002	Incorrect Access Control	Additional checks for critical logic and flow	✓
CW-003	Payable Contract	A function to withdraw ether should exist otherwise the ether will be trapped	✓
CW-004	Custom Modifier	major recheck for custom modifier logic	✓
CW-005	Divide Before Multiply	Performing multiplication before division is generally better to avoid loss of precision	✓
CW-006	Multiple Calls	Functions with multiple internal calls	✓
CW-007	Deprecated Keywords	Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions	✓
CW-008	Unused Contract	Presence of an unused, unimported or uncalled contract	✓
CW-009	Assembly Usage	Use of EVM assembly is error-prone and should be avoided or double-checked for correctness	✓
CW-010	Similar Variable Names	Variables with similar names could be confused for each other and therefore should be avoided	✓
CW-011	Commented Code	Removal of commented/unused code lines	✓
CW-012	SafeMath Override	SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking.	✓

FIXES & RECOMMENDATION

Rounding drift locks principal

```
// src/EverestVaultALT.sol:238 -- the pool's running total, rounded down
accRewardPerShare += (drip * ACC) / staked;

// src/EverestVaultALT.sol:288 -- each staker's own tally, also rounded down
u.rewardDebt = (u.amount * accRewardPerShare) / ACC;

// src/EverestVaultALT.sol:339 -- refuses to pay when the two disagree
require(pending <= allocatedRewards, "allocation mismatch");
```

The vault keeps two sets of books — one for the reward pool as a whole, one for each individual staker — and both round down to the nearest whole unit. Those roundings do not cancel out, so the stakers collectively end up owed a hair more than the pool actually holds. When the last person tries to leave, the vault sees it owes more than it has and blocks the withdrawal entirely, taking their principal down with it.

When the gap appears, pay out whatever is left instead of refusing the transaction — the shortfall is a fraction of a cent and not worth trapping a position over. Separately, add an emergency withdrawal that returns a staker's deposit and gives up their unpaid rewards, so a reward-accounting problem can never hold principal hostage again.

A one-wei staker permanently caps the vault

```
// src/EverestVaultALT.sol:238 -- divided by the amount staked, with no lower limit
accRewardPerShare += (drip * ACC) / staked;

// src/EverestVaultALT.sol:288 -- every later deposit must multiply by that number
u.rewardDebt = (u.amount * accRewardPerShare) / ACC;
```

To track rewards fairly the vault divides each payout by the total amount staked. Nothing stops that total from being a single unit — the smallest possible deposit — and dividing by one makes the running number explode to an enormous value that can never be reduced. Every future deposit has to multiply by that number, and above a certain size the calculation fails outright.

Require a sensible minimum deposit so the divisor can never be trivially small, and skip paying rewards entirely while the total staked is below a meaningful threshold. Both are a few lines and together they also close the dust-staker reward grab described above.

A dust staker collects all the rewards

```
// src/EverestVaultALT.sol:231 -- the payout is computed from the pool and the clock
uint256 drip = _previewDrip(reserve, dt); // never reads totalStaked

// src/EverestVaultALT.sol:238 -- only now is it split across whoever is staked
accRewardPerShare += (drip * ACC) / staked;
```

The size of each reward payout depends only on how much is in the reward pool and how much time has passed — it never checks whether anybody meaningful is actually staked. Whoever happens to be staked splits the full amount between them, so a single person holding a negligible deposit receives the entire day's rewards.

Do not release rewards until the total staked passes a minimum threshold; the undistributed amount simply stays in the pool and goes out later, so nothing is wasted. Alternatively, cap each payout as a percentage of what is actually staked, which bounds the return rate no matter how few people participate.

Renouncing ownership strands the fees

```
// Inherited from OpenZeppelin Ownable -- one transaction, no confirmation step
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}

// src/EverestVaultALT.sol:356 -- the only way to reach collected fees
function withdrawFees(address to, uint256 amount) external nonReentrant onlyOwner
```

The contract deliberately uses a two-step, confirm-before-it-takes-effect process for handing ownership to someone else, but it inherits a one-step function that discards ownership outright. Since only the owner can collect the fees the vault has accumulated, using it would lock those fees inside the contract permanently.

Disable that function, or require the accumulated fees to be withdrawn before it can succeed. If giving up ownership is genuinely intended as a final state, sweep the fees to the owner inside the same transaction.

AUDIT COMMENTS | Altitude

Smart Contract audit comment for a non-technical perspective

- Owner can withdraw fees
- Owner cannot block users
- Owner cannot update fees
- Owner cannot update max transaction amount
- Owner cannot pause contract
- Owner cannot mint after initial deployment
- Owner cannot burn





CONTRACTWOLF

Blockchain Security - Smart Contract Audits